

Relatório Técnico Consolidado: Projeto Pêndulo Invertido

Robô de Duas Rodas com Autoequilíbrio

Ponto de Controle 3

Artur Rodrigues de Lima
Dara Yuna Borges Fujii
Maria Gabriella Requette Marinho
Mayra Sales da Costa
Pedro de Sousa Mesquita

Centro Universitário IESB

2026

Disciplina: Automação e Robótica

Instituição: Centro Universitário IESB

1 Resumo

Este projeto consiste no desenvolvimento de um robô de duas rodas com autoequilíbrio, um sistema dinamicamente instável que representa um problema clássico de controle: o pêndulo invertido. A estratégia de controle baseia-se em um algoritmo PID (Proporcional-Integral-Derivativo) em malha fechada, processado na plataforma embarcada Arduino Uno (ATmega328P). O estágio atual do projeto consolida o planejamento detalhado, englobando a modelagem matemática, o mapeamento elétrico e a discretização do firmware. A validação ocorrerá em etapas de integração contínua, partindo de testes isolados de subsistemas em bancada até a implementação experimental em solo diante de perturbações externas.

2 Introdução

Sistemas de autoequilíbrio são amplamente aplicados em mobilidade urbana (como *Segways*), robôs de serviço e transporte de carga em ambientes logísticos. O projeto integra conceitos fundamentais da disciplina de Automação e Robótica, tais como o controle discreto (leitura de sensores e atuação em malha fechada em intervalos fixos), implementação embarcada (uso de interrupções de hardware e *timers*) e integração de sistemas ciber-físicos

(fusão de dados inerciais de acelerômetro e giroscópio). A demanda por tais sistemas reside na necessidade de veículos com alta manobrabilidade e pegada reduzida em espaços confinados, onde o controle ativo de estabilidade substitui a estabilidade passiva convencional.

O presente relatório (Ponto de Controle 3) documenta a transição do projeto da fase puramente conceitual para a materialização prática. Neste estágio, o sistema transmite maior maturidade técnica, contando com a modelagem estrutural validada em ambiente virtual (Webots), a construção de um protótipo físico preliminar em papelão e a integração elétrica entre a lógica de controle, sensores e atuadores.

3 Trabalhos Relacionados

Para o embasamento deste projeto, foram analisadas implementações acadêmicas e open-source, das quais extraímos os resultados e mapeamos limitações a serem superadas:

- **Arduino Two-Wheel Self-Balancing Robot (Project Hub):** Aborda a montagem e estruturação básica de um robô equilibrista.
 - *Resultados:* Demonstração funcional de equilíbrio utilizando peças de baixo custo.
 - *Limitações Identificadas:* O código apresenta uma estrutura bloqueante (*polling* contínuo), o que gera *jitter* na leitura inercial e prejudica a frequência exata do cálculo PID. Nosso projeto resolve isso adotando interrupções de hardware.
- **Simple Two-wheel Self-Balancing Robot Implementation:** Implementação de um sistema ciber-físico utilizando motores de passo para transladar a base.
 - *Resultados:* Robô capaz de compensar perturbações e seguir comandos via Bluetooth.
 - *Limitações Identificadas:* O uso de motores de passo encarece o sistema e aumenta a complexidade do driver de potência. Optamos por motores DC com caixa de redução e controle PWM para otimizar custo e viabilidade.
- **Control Strategies for Two-Wheeled Systems (MDPI):** Explora métodos avançados como FOPID e redes neurais.
 - *Resultados:* Sugere o uso de derivadas fracionárias para respostas não-lineares suaves.
 - *Limitações Identificadas:* Altíssima carga computacional, inviável para processadores de 8-bits como o ATmega328P. A solução adotada em nosso projeto foi manter o PID clássico, mas utilizar a técnica de "Derivada na Medição" para mitigar solavancos (*Derivative Kick*), alcançando estabilidade similar com menor custo de processamento.

4 Fundamentação Teórica

4.1 Problema de Engenharia

O design estrutural do robô é baseado no conceito de um pêndulo invertido sobre rodas coaxiais. A variável de interesse é o ângulo de inclinação vertical no eixo Pitch (θ).

Estabelecemos um *setpoint* próximo a 180° em relação ao eixo gravitacional, compensando assimetrias físicas do chassi. O sistema deve atuar contra perturbações como obstáculos frontais, irregularidades no solo, ruído eletromagnético nos sensores e forças externas (toques manuais).

4.2 Requisitos Técnicos Consolidados

Os requisitos do projeto foram quantificados para permitir a validação experimental e o ajuste de hardware.

Descrição do Requisito	Justificativa Técnica	Métrica / Critério de Aceitação	Valor Alvo Aceitável
Estabilidade Dinâmica	Manter o equilíbrio sem oscilação excessiva que leve à saturação dos motores.	Erro angular máximo (Pitch) em repouso após sintonia PID.	$\pm 3^\circ$
Tempo de Resposta (Loop)	Reagir a perturbações antes que a gravidade vença o torque mecânico máximo.	Frequência de execução do firmware de controle.	≥ 100 Hz (Ciclo ≤ 10 ms)
Tensão Operacional	Garantir que o driver receba energia suficiente para acionar os motores nominais.	Tensão estável entregue aos motores em carga.	~ 6.0 V
Autonomia de Energia	Evitar <i>reset</i> do microcontrolador por queda de tensão em picos de corrente de motor.	Tempo de operação contínua sem afundamento (<i>Brownout</i>).	≥ 15 minutos
Segurança (<i>Fail-Safe</i>)	Impedir que os motores continuem girando caso o robô caia definitivamente no chão.	Ângulo limite para corte automático de potência.	$> 150^\circ$ ou $< 200^\circ$

4.3 Modelagem Preliminar e Teoria de Controle

O sistema é inicialmente modelado como um pêndulo invertido. A dinâmica física (não-linear) que rege a queda do robô, relacionando o ângulo θ e o deslocamento x , pode ser simplificada na seguinte equação diferencial:

$$l\ddot{\theta} - g \sin \theta = \ddot{x} \cos \theta$$

Trabalhar com funções trigonométricas no código do microcontrolador é computacionalmente inviável. Para contornar isso, linearizamos o sistema em torno do ponto de equilíbrio ($\theta \approx 0$). Matematicamente, para ângulos muito pequenos, assume-se que $\sin \theta \approx \theta$ e $\cos \theta \approx 1$.

Ao aplicar a **Transformada de Laplace**, convertemos as equações diferenciais complexas do domínio do tempo para equações algébricas simples no domínio da frequência (s). Isso nos permite desenhar o **Diagrama de Blocos de Controle** abaixo:

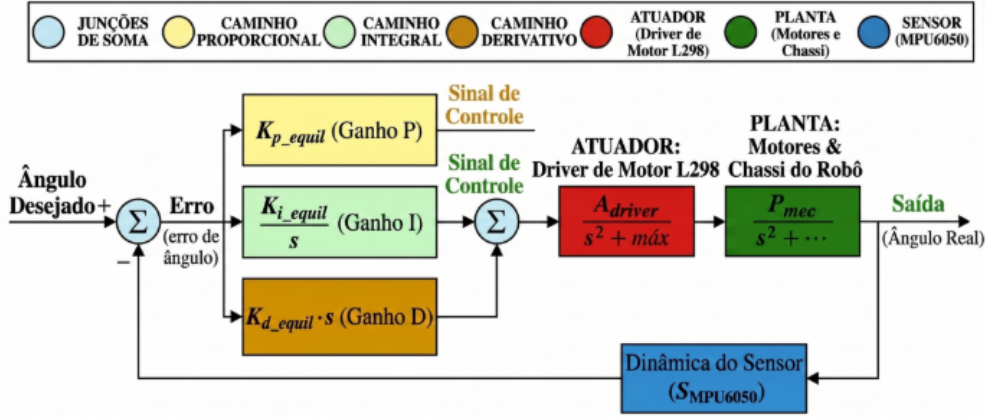


Figura 1: Diagrama de Blocos de Controle (PID)

A representação acima é a topologia matemática exata da função de transferência em malha fechada do projeto, baseando-se em princípios de Ogata:

1. **Isolamento da Lógica de Controle $C(s)$:** O controlador PID processa o sinal de erro: $C(s) = K_p + \frac{K_i}{s} + K_d s$.
2. **Modelagem da Planta e Atuador $G(s)$:** Modela o Driver L298 e os Motores como funções de transferência contendo polos ($s^2 + \dots$), prevendo a inércia mecânica do motor.
3. **Garantia de Estabilidade:** Ajustando empiricamente os ganhos (K_p , K_i , K_d), garantimos matematicamente que as raízes da equação característica fiquem no semiplano esquerdo do plano complexo.

5 Metodologia

A metodologia de desenvolvimento adotou uma abordagem incremental, dividindo o projeto físico e lógico entre a equipe para otimização de tempo:

- **Mecânica e Estrutura (Dara Yuna):** Modelagem virtual das peças no *Webots* para validação física, seguida da construção de um protótipo com hastes de madeira e papelão.
- **Elétrica (Maria Gabriella):** Execução do cabeamento de potência, fixação do driver e soldagem dos capacitores EMI para proteção.
- **Hardware e Lógica de Software (Artur e Pedro):** Migração para a plataforma *PlatformIO*, adaptação do código C++ para isolamento de pinos lógicos no L298N e validação do barramento I2C.
- **Sintonia e Validação (Mayra e Equipe):** Utilização da estratégia de "Teste Suspenso" monitorando os valores do *Serial Plotter* para calibração.

6 Desenvolvimento

6.1 Arquitetura do Sistema

O fluxo de informações e energia ocorre de maneira distribuída.

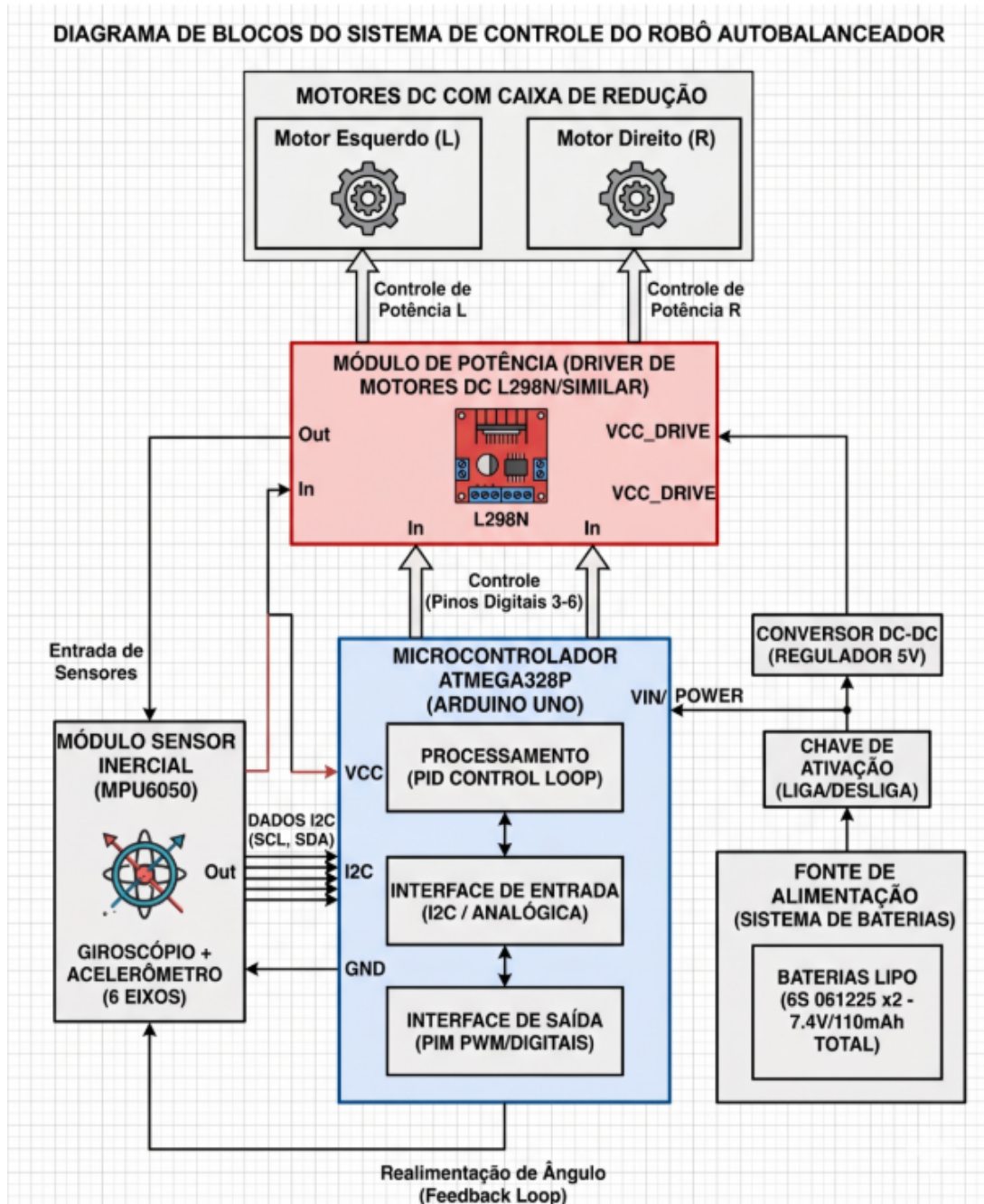


Figura 2: Diagrama de Blocos do Sistema de Hardware

O sistema conta com o Microcontrolador (Arduino Uno - ATmega328P) processando o

sinal; o Sensor Inercial (MPU6050) utilizando seu coprocessador DMP para interrupções I2C; e os Atuadores recebendo interface de isolamento pelo Driver L298N.

6.2 Hardware Implementado, Materiais e Componentes

A alimentação foi projetada em duas células Li-Ion em série ($3.7V + 3.7V = 7.4V$).

Para garantir o isolamento adequado e evitar quedas (*brownouts*), adotamos a topologia em cascata: a bateria fornece energia bruta ao L298N, que possui uma queda térmica de $\sim 1.4V$ e entrega $\sim 6.0V$ perfeitos aos motores (evitando sobrecarga). Simultaneamente, o regulador interno do L298N fornece 5V lógicos ao Arduino, que por fim regula 3.3V ultraestáveis para o MPU6050.

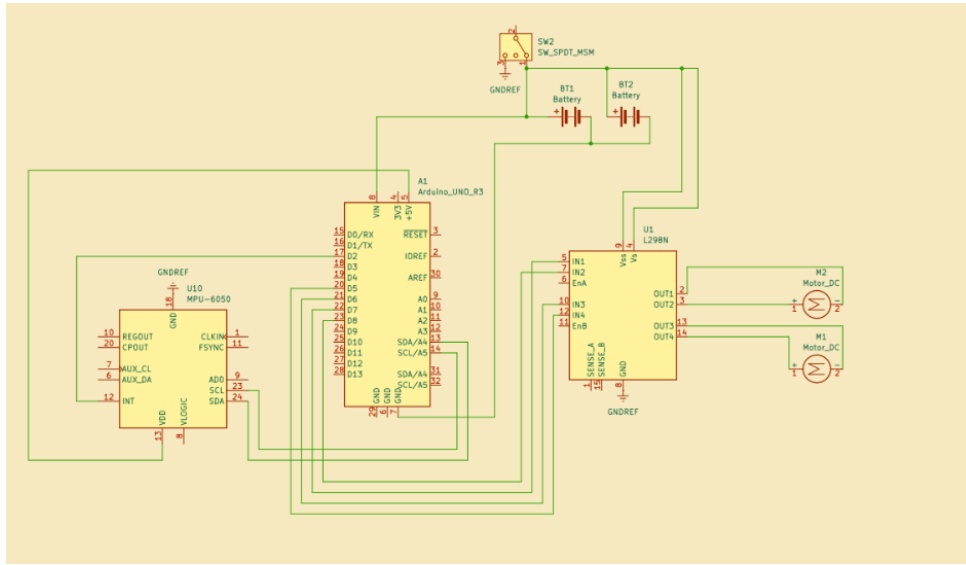


Figura 3: Esquemático Preliminar de Integração Eletrônica

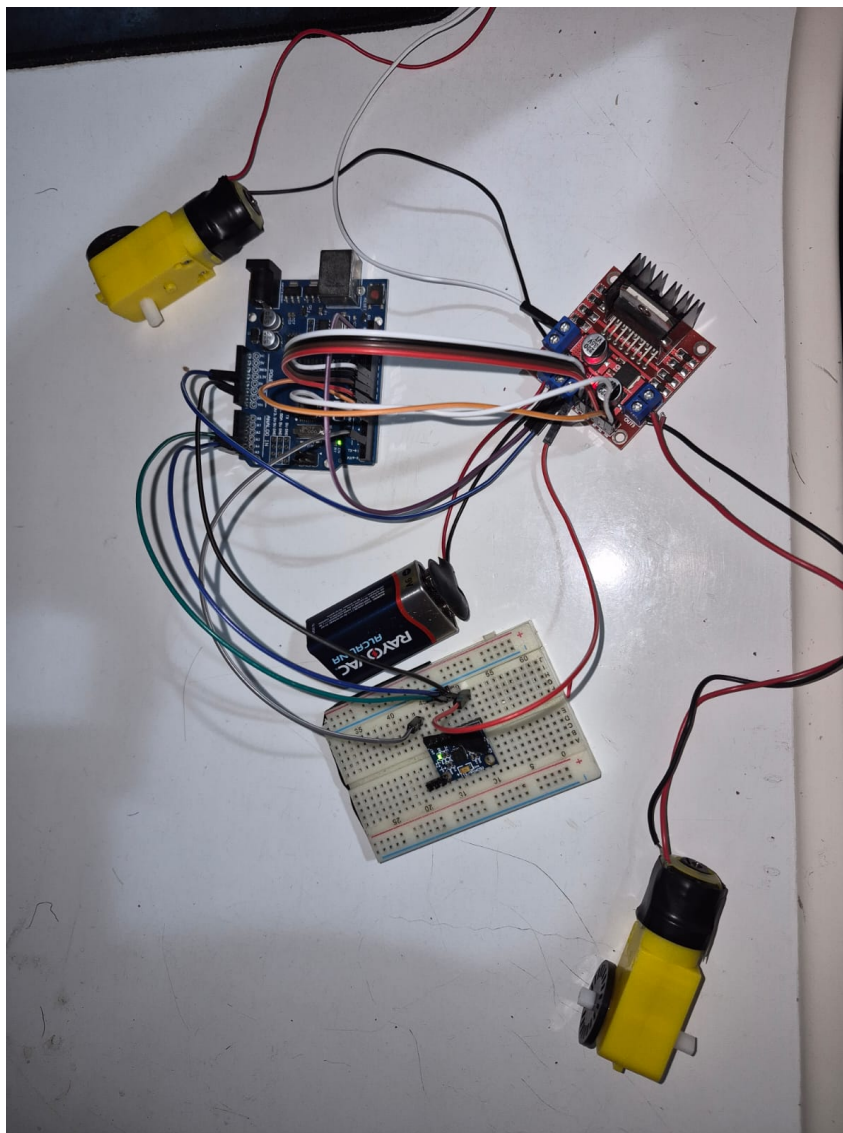


Figura 4: Circuito lógico e de potência integrados na bancada.

A tabela abaixo compila a análise de materiais implementada no protótipo físico:

Componente	Qtd	Justificativa Técnica	Custo Est.
Arduino Uno R3	1	Núcleo de processamento do algoritmo PID e lógica de interrupções.	R\$ 65,00
MPU6050	1	Sensor inercial com módulo DMP para cálculo preciso de inclinação via I2C.	R\$ 25,00
Driver L298N	1	Atua na transição de potência, convertendo lógica PWM em força eletromecânica.	R\$ 20,00
Motores DC (TT 6V)	2	Fornecem o torque necessário para as rodas coaxiais restaurarem a base de centro de massa.	R\$ 30,00
Rodas (65mm)	2	Interface de tração emborrachada para garantir atrito na translação.	R\$ 15,00
Baterias Li-Ion 18650	2	Fonte de 7.4V com alta taxa de descarga (evitando afundamento de tensão).	R\$ 50,00
Suporte 18650 Duplo	1	Centraliza o peso no andar inferior do chassi.	R\$ 10,00
Capacitor 100nF	4	Soldados aos motores para filtragem (Passa-Baixa) de ruído eletromagnético EMI.	R\$ 2,00
Chassi Físico	-	Material estrutural validado provisoriamente em papelão para testes preliminares.	-

6.3 Firmware e Estrutura de Código

A equação matemática contínua do PID foi discretizada via software utilizando a biblioteca oficial PID_v1.h. O firmware engloba:

A. Arquitetura Não-Bloqueante (Event-Driven):

Em vez de sobrecarregar o *loop* principal (*polling*), o Arduino aguarda interrupções de hardware (Pino 2).

```

1 // Aguarda a interrupcao do MPU6050 indicando que ha dados no buffer
2 while (!mpuInterrupt && fifoCount < packetSize) {
3     // Arquitetura nao-bloqueante: a frequencia de 100 Hz e garantida
4     pid.Compute();
5
6     // Fail-safe de queda
7     if (input > 200 || input < 150) { Stop(); }
8     else { Atuacao_Motores(); }
9 }

```

Listing 1: Exemplo de Arquitetura Não-Bloqueante

B. Separação de Responsabilidade (L298N em 6 Pinos):

O controle de motor no código foi reescrito. Agora, a lógica de sentido magnético é feita com portas digitais puras (IN1 a IN4), enquanto a intensidade (velocidade calculada pelo

PID) é injetada estritamente nas portas ENA/ENB do driver, evitando curtos lógicos na reversão.

C. Mitigação Integral (*Anti-Windup*) e Derivada:

A função `pid.SetOutputLimits(-255, 255)`; trava o acúmulo da integral. A derivada é calculada apenas observando a variação da medição do sensor para mitigar o *Derivative Kick*.

6.4 Integração Física e Modelagem 3D

A concepção mecânica adota o chassi em três andares, abaixando o centro de massa. O andar inferior retém baterias e motores, o intermediário retém as placas lógicas, e o topo segura o MPU6050 (otimizando o arco inercial de leitura).

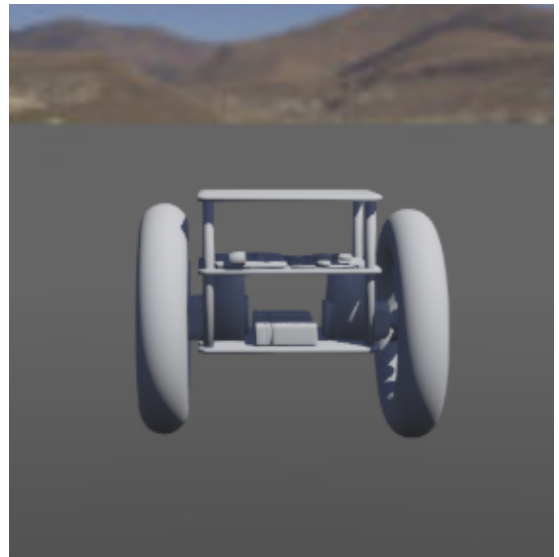
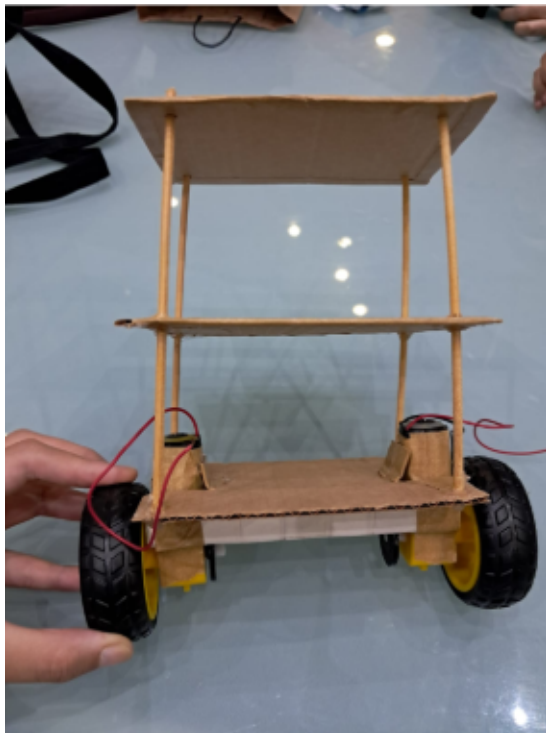


Figura 6: Modelagem 3D no Webots.

Figura 5: Protótipo preliminar físico.

Nota: Imagens adicionais de modelagem e montagem podem ser vistas na Seção Anexos.

6.5 Estratégia Experimental

A validação física segue o fluxo contínuo. Atualmente, os testes adotam a "Sintonia Suspensa": o robô é segurado com as rodas no ar enquanto inclinamos a estrutura. Validamos a polaridade lógica comparando os valores *input* (180°) e *output* do PID (-255 a 255) diretamente no Monitor Serial (115200 bps).

7 Resultados e Discussão

As avaliações preliminares em bancada demonstraram que o barramento I2C operou sem travamentos ou falhas na FIFO. A leitura inercial converte com sucesso o desvio gravitacional em comandos elétricos via código. Durante o desenvolvimento, as dificuldades técnicas foram mapeadas e corrigidas:

Dificuldade	Impacto	Ação Corretiva
Ruído Eletromagnético (EMI)	Interferência nas linhas I2C, travando o MPU6050.	Soldagem de capacitores cerâmicos de 100nF nos motores.
Queda de Tensão (<i>Brownout</i>)	Reset do Arduino devido à alta corrente de pico.	Planejamento de uso de baterias Li-ion 18650 e cascata de proteção.
Atraso Sensorial (<i>Jitter</i>)	Invalida a premissa de tempo constante ($dt = 10ms$) do PID.	Uso de Interrupção de Hardware (Pino 2) combinada ao DMP.
Sintonia do PID	Dificuldade em achar os ganhos K_p , K_i e K_d ideais.	Adoção da metodologia de "Teste Suspenso" para calibração.

Ao tentarmos testar temporariamente o sistema do PC3 com baterias puramente alcalinas (9V convencionais), ocorreu o comportamento limitante previsto de *brownout* leve, onde os sensores leem, mas os atuadores não possuem corrente para girar o torque nominal sob o peso da placa. Isso corrobora a exigência das baterias Li-Ion mapeadas.

8 Conclusão

O estágio real do projeto marca a materialização física da proposta conceitual (PC2) em um protótipo físico integrado (PC3). O robô em papelão, aliado à validação do Webots, **comprovou a eficácia** estrutural da divisão de andares. Eletronicamente, o circuito lê os limites angulares adequadamente.

As pendências para o PC4 consistem em substituir a alimentação provisória pelas células recarregáveis de alto rendimento. Feito isso, o projeto avança imediatamente para o solo, buscando a sintonia de ganhos finais para a estabilidade autônoma final.

9 Cronograma Atualizado de Execução

Tarefa de Desenvolvimento (Rumo ao PC3/PC4)	Responsável	Prazo
1. Mecânica: Impressão final do chassi em PLA / montagem.	Dara Yuna	Semana 1
2. Elétrica: Soldagem dos capacitores EMI e fixação dos drivers.	Maria Gabriella	Semana 1
3. Hardware: Teste de subsistemas isolados na bancada.	Artur e Pedro	Semana 2
4. Software: Refatoração PID para L298N de 6 pinos.	Pedro	Semana 2
5. Sintonia do Controle: Ajuste dos parâmetros K_p , K_i , e K_d via <i>Serial Plotter</i> (Robô Suspensão).	Mayra e Pedro	Semana 3
6. Validação Experimental: Teste dinâmico em solo, validação da autonomia e perturbações físicas.	Equipe	Semana 4

10 Referências

- ARDUINO. **Arduino Uno R3 Datasheet**. [S. l.], 2023. Disponível em: <https://docs.arduino.cc/rev3>. Acesso em: 30 abr. 2026.
- ASCENTO: A Two-Wheeled Jumping Robot. [S. l.], 2023. Disponível em: <https://www.ascento.cc/>. Acesso em: 26 abr. 2026.
- CHAN, R. P. M. et al. **Mathematical Modeling of a Two Wheeled Robotic Base**. [S. l.: s. n.], 2013.
- DA SILVA, J. S. **O sistema dinâmico pêndulo invertido: modelagem e projeto de controladores por simulação computacional**. [S. l.: s. n.], 2018.
- HUANG, J. et al. Design and dynamic analysis of jumping wheel-legged robot in complex terrain environment. **Applied Sciences**, [S. l.], v. 10, n. 21, 2020.
- INVENSENSE. **MPU-6050 Product Specification**. Revision 3.4. San Jose: InvenSense Inc., 2013.
- LI, J. et al. Design and implementation of two-wheeled self-balancing mobile robot control system based on STM32-MAT and Android. **Journal of Measurements Engineering**, [S. l.], v. 8, n. 2, p. 55-66, 2020.
- NASIR, A. N. K. et al. Genetic Algorithm-based Control of a Two-Wheeled Self-Balancing Robot. **Journal of Intelligent & Robotic Systems**, [S. l.], v. 60, n. 3, p. 345-362, 2010.
- OGATA, K. **Engenharia de controle moderno**. 5. ed. São Paulo: Pearson, 2010.

- PATHAK, K. et al. Recent developments in the applications of two-wheeled robots: a review. **International Journal of Dynamics and Control**, [S. l.], 2019.
- PAVLESKI, M. **Arduino Two-Wheel Self-Balancing Robot**. Arduino Project Hub, 2023.
- SANTOS, A. L. **Sistema de controle de pêndulo invertido**. [S. l.: s. n.], 2015.

11 Anexos



Figura 7: Visualização da disposição dos eixos no protótipo estrutural.

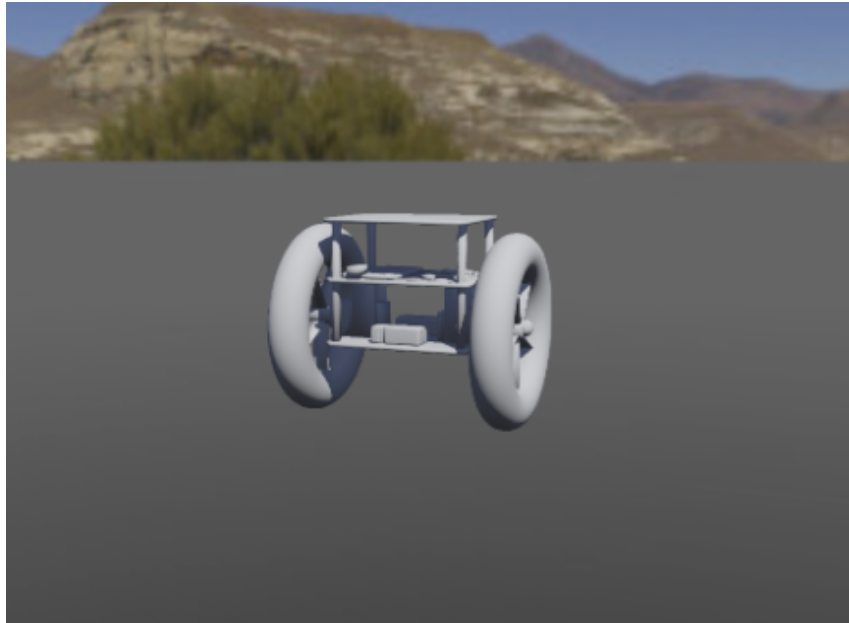


Figura 8: Detalhamento de eixos e rodas dimensionadas no simulador Webots.